

REKOLT Planters' Cooperative Produce Tracker

Objective 4

This document covers the scope, requirements, a noun-verb analysis, a UML class diagram, an activity diagram, pseudocode for the net payment calculation, a data dictionary, and a rationale for three design decisions.

Functional requirements are numbered FR1–FR17 (section 2) and non-functional requirements NFR1–NFR7 (section 3). Later sections cite these numbers in parentheses, e.g., “(FR9)”, to refer to functional requirement 9.

1. Scope statement

This project builds a single-user, console-driven Java application for REKOLT Planters' Cooperative. It records one season's produce deliveries, computes each delivery's net payable using the fixed rules in section 2 of the specification, answers on-screen questions about the season so far, and generates one Microsoft Word document season report combining a per-member statement and a reconciling closing total. All delivery data lives in memory for the lifetime of one run only, except the generated report and its run log, which are written to disk.

Assumptions

- A. Only one season is tracked per program run; there is no cross-season history.
- B. All money is in Mauritian Rupees (MUR); no currency conversion is required.
- C. The program is operated by one user (the treasurer or a delegate) at a time via the console; there is no concurrent or multi-user access.
- D. Produce codes, base prices, categories and grade boundaries are fixed for the whole season, as given in section 2, and do not change while the program runs.
- E. A member identifier uniquely and permanently identifies one planter; the same ID is never reused for two different people within a season.

2. Functional requirements

- 1. The system shall accept exactly one of MZE, BNS, POT, TEA as a produce code for a delivery, case-insensitively, and reject any other value with a re-prompt.
- 2. The system shall accept a member identifier only in the form "M-" followed by exactly four digits, and reject any other format with a re-prompt.
- 3. The system shall not accept an empty member name or whitespace-only input.
- 4. The system shall accept a delivery mass strictly greater than 0 kg and no more than 5000 kg, rejecting non-numeric, zero, negative, or over-limit values.
- 5. The system shall accept a quality score as a whole number from 0 to 100 inclusive, rejecting decimals, negatives, or values above 100.
- 6. The system shall accept a week number as a whole number from 1 to 20 inclusive.
- 7. The system shall derive a grade of A, B, C, or REJECT from the quality score using the exact boundaries: A for 85–100, B for 70–84, C for 50–69, REJECT below 50.
- 8. The system shall compute net payable for a non-REJECT delivery as the five-step calculation in section 2, evaluated in that exact step order.
- 9. The system shall set net payable, commission, and transport levy to zero for a REJECT delivery, while still recording its mass towards volume statistics.
- 10. The system shall allow the user to record any number of deliveries in one run of the program via a repeating menu.
- 11. The system shall display, on demand, the total net payable owed to each distinct member seen so far this season.
- 12. The system shall display, on demand, a weekly grid of mass delivered per produce code.
- 13. The system shall display, on demand, the deliveries ranked by net payable value, excluding REJECT deliveries from that ranking.
- 14. The system shall look up a delivery by its identifier and report clearly whether or not it was found, without raising an unhandled exception either way.

15. The system shall generate one Microsoft Word document containing one section per member (each starting on a new page) and a closing section whose totals reconcile with the sum of the member sections.
16. The system shall append one timestamped line to a run log each time the season report is generated.
17. The system shall never terminate with an uncaught exception as a result of user-typed input.

3. Non-functional requirements

1. All monetary values shall be held as double-precision, unrounded, through every calculation step, and rounded to two decimal places only at the point of display or when written to the report.
2. The season's delivery data shall exist only in memory for the lifetime of one run and shall not be persisted to or read from any file, other than the generated season report and run log.
3. Every console prompt shall re-request input, with a short stated reason, rather than allow the program to crash on invalid input.
4. The repository shall keep source code, documentation, and generated output in three separate top-level folders — src, docs, and output — with none nested inside another.
5. Source code shall be maintained under git with feature-branch development, and each objective's work merged into main via a non-fast-forward merge.
6. The system shall run on JDK 17 or later with no external runtime dependencies beyond the declared Maven dependencies.
7. The season report file shall be written using try-with-resources so the output stream is always closed, even if writing fails partway through.

4. Noun-verb analysis

Nouns drawn from the scenario and the payment rules: cooperative, planter/member, clerk, treasurer, committee, delivery, slip, produce (maize, beans, potatoes, tea), quality score, grade, mass, price, commission, transport levy, net payable, season, week, statement, report, signature, run log.

Classes kept

Noun	Class	Why kept
planter / member	Member	Has persistent identity (ID, name) and behaviour (total payable, its own report section).
delivery	Delivery	The core recorded event: has its own identity, data, and behaviour (net payable, one report line).
produce, maize, beans, potatoes, tea, category	Produce (abstract) + CerealProduce, PerishableProduce, CashCropProduce	Behaviour (the category multiplier) genuinely differs by category — a natural fit for polymorphism.
quality score → grade	Grade (enum)	A small fixed set of named values, each carrying its own multiplier.
price	PriceListEntry (Objective 3) → folded into Produce subclasses (Objective 5)	Started as a data row; once Produce exists, the price list becomes an array of Produce instances instead, so the multiplier lives with the behaviour that uses it.
season	SeasonStore	Groups this run's members and deliveries; owns the collections built in Objective 3.

Classes rejected (three, with reasons)

Noun	Why rejected
Clerk	The clerk's job weighing produce, giving a quality score, writing a slip is entirely replaced by validated console input. There is no data or behaviour left for a Clerk object to hold; it would just be a synonym for "the user".
Committee	The committee asks the treasurer questions at season's end; they never interact with the system directly. There is nothing for a Committee class to store or do.
Slip	The paper slip's entire purpose of recording one delivery's details is superseded by Delivery, which holds the same information as validated, structured data. Keeping both would duplicate one record under two names.

5. UML class diagram

Notation: hollow-triangle arrows show inheritance (Produce's subclasses) and, dashed, interface realisation (Payable, Reportable). Plain arrows with multiplicities show association. A hollow diamond shows aggregation (SeasonStore holds Members, which can exist independently of one run). A filled diamond shows composition (a Delivery only has meaning as part of one Member's recorded history). Abstract class names and abstract methods are italicised; static members are underlined, per UML convention. Every relationship line is labelled with what it means; a notation key sits in the top-right corner.

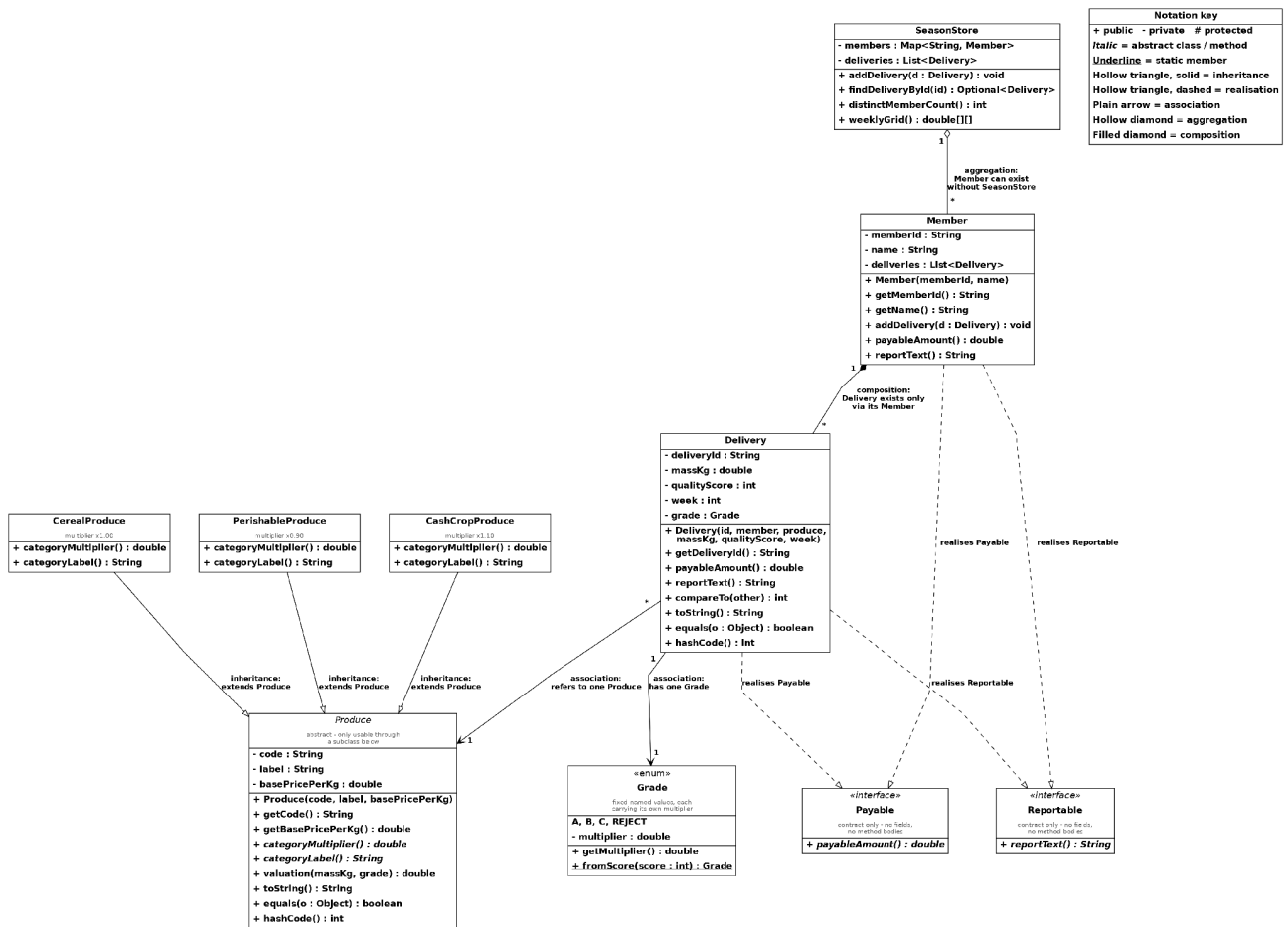


Figure 1. Target class design for Objectives 5 and 6.

6. Activity diagram — record a delivery

This traces one call to “Record a delivery”, from validated input through to the REJECT branch and the final receipt, matching the pseudocode in section 7.

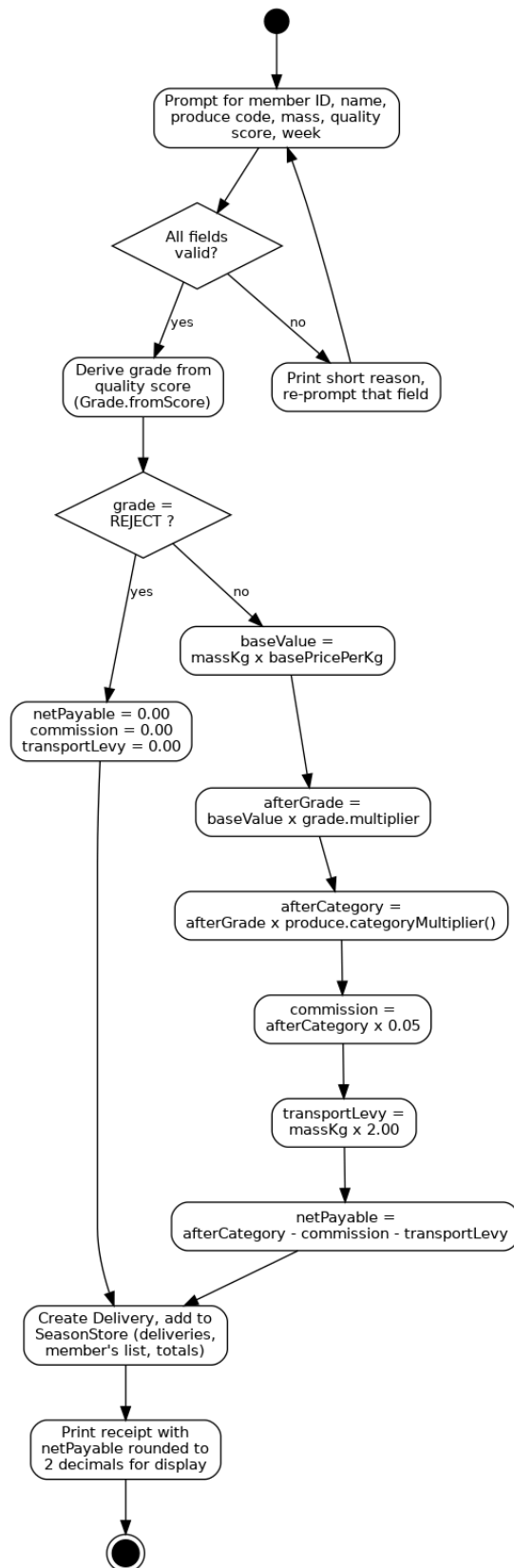


Figure 2. Activity diagram for recording one delivery.

7. Pseudocode — net payment, including the REJECT case

```
FUNCTION computeNetPayable(massKg, produce, grade)
  baseValue      <- massKg * produce.basePricePerKg
  afterGrade     <- baseValue * grade.multiplier
  afterCategory  <- afterGrade * produce.categoryMultiplier()

  IF grade = REJECT THEN
    RETURN 0.00
  END IF

  commission      <- afterCategory * COMMISSION_RATE      // 0.05
  transportLevy   <- massKg * TRANSPORT_LEVY_PER_KG       // 2.00
  netPayable      <- afterCategory - commission - transportLevy

  RETURN netPayable
END FUNCTION
```

A REJECT delivery still reaches this function and is still recorded afterwards with its mass counted towards the weekly volume grid (FR9) — only its payable amount, commission and levy are forced to zero.

8. Data dictionary

Field	Type	Unit	Range / domain	Validation rule
deliveryId	String	—	"D-" + digits	Assigned by the system, never user-entered.
memberId	String	—	M-0000 to M-9999	Must match M-\d{4} exactly (FR2).
memberName	String	—	any non-blank text	Rejected if empty or whitespace-only (FR3).
produceCode	String	—	MZE, BNS, POT, TEA	Case-insensitive match to one of the four codes (FR1).
massKg	double	kg	0 < mass ≤ 5000	Rejected if non-numeric, ≤0, or >5000 (FR4).
qualityScore	int	score	0 to 100	Whole number only; rejected if outside range (FR5).
week	int	week no.	1 to 20	Whole number only; rejected if outside range (FR6).
grade	Grade (enum)	—	A, B, C, REJECT	Derived from qualityScore; never entered directly (FR7).
basePricePerKg	double	MUR/kg	25.0 to 90.0	Fixed per produce code, from section 2 rule 1.
categoryMultiplier	double	—	0.90, 1.00, 1.10	Fixed per category, from section 2 rule 3.
commissionRate	double	—	0.05 (fixed)	Applied only to non-REJECT deliveries (FR8, FR9).
transportLevyPerKg	double	MUR/kg	2.00 (fixed)	Applied only to non-REJECT deliveries (FR8, FR9).
netPayable	double	MUR	≥ 0	Unrounded until display; rounded to 2 dp only then (NFR1).

9. Rationale — three decisions defended

9.1 Payable and Reportable as shared single-method interfaces

Delivery and Member both need to expose a payable amount and a piece of report text (FR15), but they compute those very differently. Rather than giving each class its own differently-named methods, both implement a common Payable and Reportable interface (`payableAmount()`, `reportText()`). This satisfies Objective 5's requirement that an interface be implemented "by more than one class", and it lets ReportService iterate a single `List<Reportable>` when building the season report, calling `reportText()` without ever needing instanceof or a cast to tell a Delivery from a Member. Rejected alternative: giving each class its own ad hoc method names (e.g. `Delivery.netPayable()` and `Member.totalOwed()`) was rejected, since the report-writing code would then need to branch on type to know which method to call exactly the kind of downcasting Objective 5 rules out.

9.2 The price list becomes an array of Produce, not a separate data row

Objective 3 held the price list as an array of `PriceListEntry`, plain data rows read by a linear search. Once Produce and its subclasses exist, that design is replaced: the "price list" becomes an array of four Produce instances (two `CerealProduce`, one `PerishableProduce`, one `CashCropProduce`), each already knowing its own base price and its own category multiplier through `categoryMultiplier()`. Rejected alternative: keeping `PriceListEntry` alongside the new Produce hierarchy was rejected, because the category multiplier would then exist in two places at once as data on the entry and as behaviour on the subclass which is a direct route to the two drifting out of sync, and it would force PaymentService back to branching on produceCode strings instead of calling `produce.categoryMultiplier()` polymorphically.

9.3 Member composes Delivery, rather than a flat, unrelated list

The design gives Member its own deliveries field, making the relationship composition: a given Delivery only has meaning as part of one member's recorded history, and per NFR2 the season's data doesn't outlive one run anyway, so tying a Delivery's lifecycle to its Member is not an overly restrictive choice. It also directly serves FR15 and the Objective 3 requirement that a HashSet of distinct member identifiers "tells the report how many member sections it needs" with composition, that count is simply `members.size()`. Rejected alternative: keeping one flat, ownerless `List<Delivery>` with member identifiers as plain string fields (the Objective 2/3 approach) was considered, but rejected for the design going forward, since grouping by member would then always require a re-scan or an auxiliary map rather than being a property of the Member object itself.